

Lab 2 Worksheet

Machine Model, Pointers/References, Call Stack, UTF Tutorial

Lab is worth 5 points total

1. This worksheet is worth 4 points. When you reach the end of a checkpoint, let your lab instructor know. They will check your work for correctness and give you points for your work. You must be present for each checkpoint discussion with your lab instructor to earn those points.
2. Earning credit for lab requires attending the lab exercises in-person. Staff will take attendance during lab. You must be present in lab to earn the 1 attendance point.

Logistics

One group member should create an editable copy of this worksheet to fill out and share the document with all other group members. (*File > Make a Copy* in the menu.) Please do not change the formatting of the document or which exercises appear on which slides. We use the placement of questions on each page to assist with grading.

Group Members

Add your names to the table below. You are each responsible for participating in the group exercises and for ensuring others are included.

Cassandra Liu
Arjun Kapur
Chelsea
Andy Hou

2.1.0 Warm up Questions

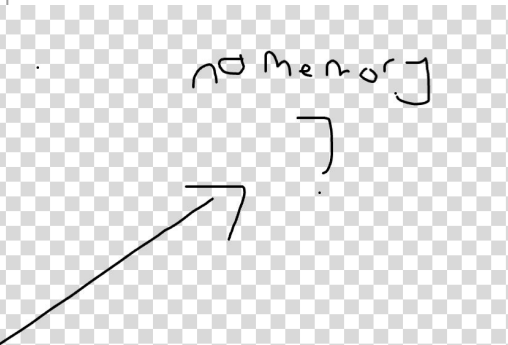
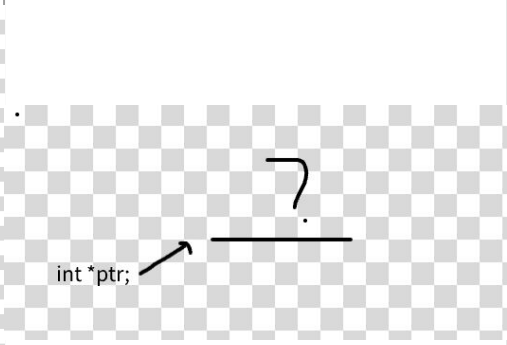
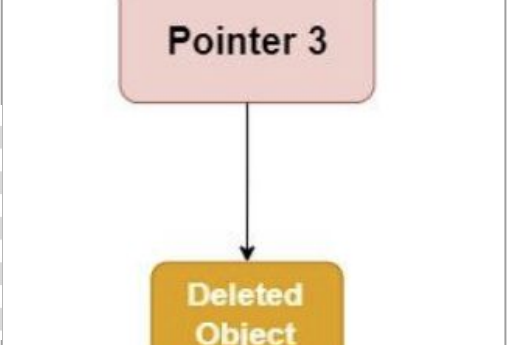
Let's get to know your group some more! What's something everyone in your group has in common? The more specific or unique the better!

We have all been outside the country.

As a group, discuss these questions together.

Question	Your answer
What are the main differences between references and pointers?	A reference is another name for the same object (same value and same address) Pointers store the memory address of another variable.
How are the & and * operators used to go back and forth between pointers and objects?	The &(gets the memory) goes from object to pointer, the * goes from pointer to object (gets the value)
Why are activation records for functions stored on a stack and not some other structure?	Last in first out, this makes the stack the most computationally efficient way to store activation records

Come up with an image or meme to represent each of the following.

A null pointer	An uninitialized pointer	A dangling pointer
		

2.1.1 Teamwork

This exercise is intended to help you and your group members build a collective understanding of teamwork, as well as a set shared values and norms for your EECS 280 lab work.

Each group member should come up with 4 words that they feel describe effective team work. Write your four words down separately - do not enter them in the table below until everyone has had a chance to finish. Then, take turns adding your words to the table below, sharing a brief explanation for why you feel it characterizes effective teamwork. If your group has fewer than 4 people, you may leave some columns blank.

name	Chelsea	Arjun	Cassandra	Andy
word 1	communication	Hard work	Accountability	Collaborative
word 2	Teach others	passion	Respect	polite
word 3	listen	determination	Coordination	helpful
word 4	feedback	Support	Reliability	responsibility

Now, work together to select 4 words from your larger set as representatives, based on your individual perspectives and discussion.

communication	support	collaborate	accountability
---------------	---------	-------------	----------------

Finally, your lab staff will stop by to chat with you about this exercise (and others!) during checkpoint 1. They'll share a couple points of feedback or perspective. Write a brief summary of your discussion on teamwork below.

Accountability is very important, everyone should remain accountable for the work that they do. Also think about being inclusive and open minded while working together

2.1.2 Pointers, References, and Memory Diagrams

We'll use Lobster (lobster.eecs.umich.edu) for this exercise. (Note - if you'd like to save a copy of your work on this or other exercises, log in to Lobster and click the "Make a personal copy" button after loading each exercise. Then Lobster will autosave to the cloud.)

Select "Course Exercises" from the left sidebar. Open the "Lab02_mem_1" exercise. You should see some starter code. Click the "Memory Diagram" switch (don't click "Simulate") and you should see an instant memory diagram, which will auto-update as you write code.

Initially, you should see a diagram like the one at right. Modify the code to initialize the program to target state #1, shown at right. Paste your modified code below.

```
int main() {
```

```
    Int x = 3;  
    Int y = 7;  
    Int &z = x;  
    Int *p1 = &z  
    Int *p2 = &y;
```

```
}
```

Beginning from your code above (in state #1), add code to reach target state #2, **without** directly using either of the variables `x` or `y`. Paste the *additional* code below.

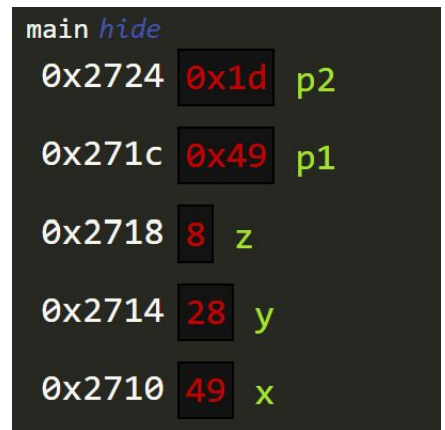
```
int main()
```

```
    ... // previous code from state #1
```

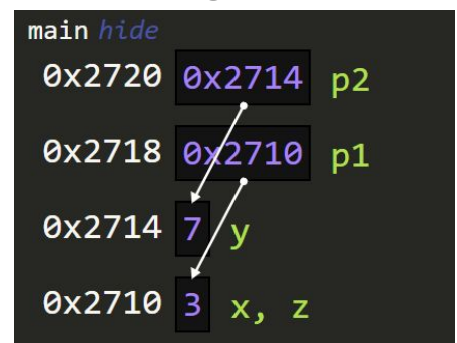
```
    *p1 = *p2;  
    p2 = p1;
```

```
}
```

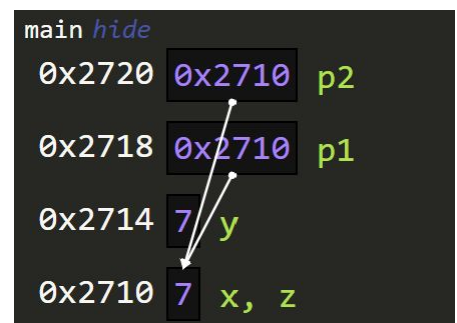
Initial State



Target #1



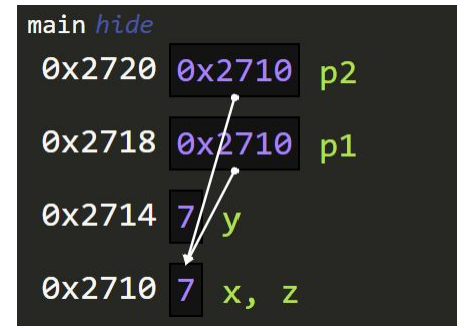
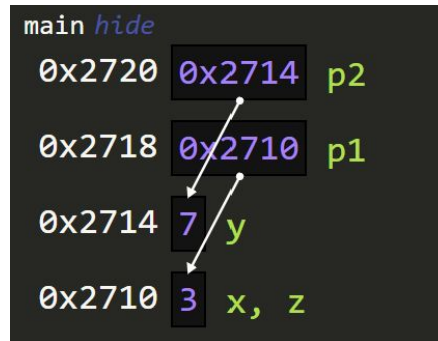
Target #2



2.1.2 Pointers, References, and Memory Diagrams (Continued)

Consider again the task of writing code to adjust the state of memory from Target #1 to Target #2.

Which of the code snippets below would successfully do this? Write "correct" or "incorrect" in each box.

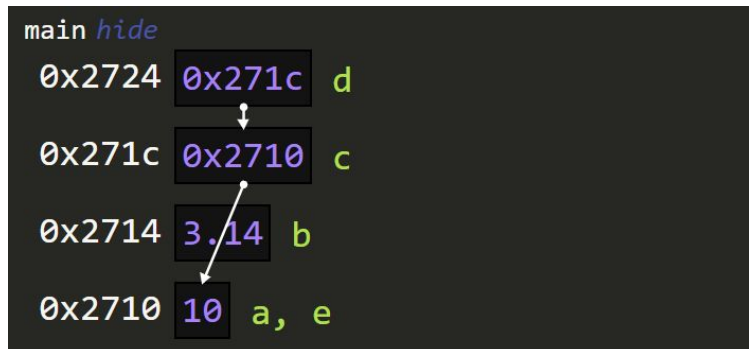


<pre>*p1 = *p2; p2 = p1;</pre>	<pre>*p2 = z; z = 7;</pre>	<pre>&p2 = &p1; *p2 = 7;</pre>	<pre>p2 = &z; *p1 = 7;</pre>
Correct	incorrect	incorrect	Correct

2.1.3 Pointer Types and Operators

Consider the code below and the corresponding memory diagram.

```
int main() {  
    int a = 10;  
    double b = 3.14;  
    int *c = &a;  
    int **d = &c;  
    int &e = *c;  
}
```



For each expression below, indicate its type and the value it would have if printed cout. We've filled in a few examples for you.

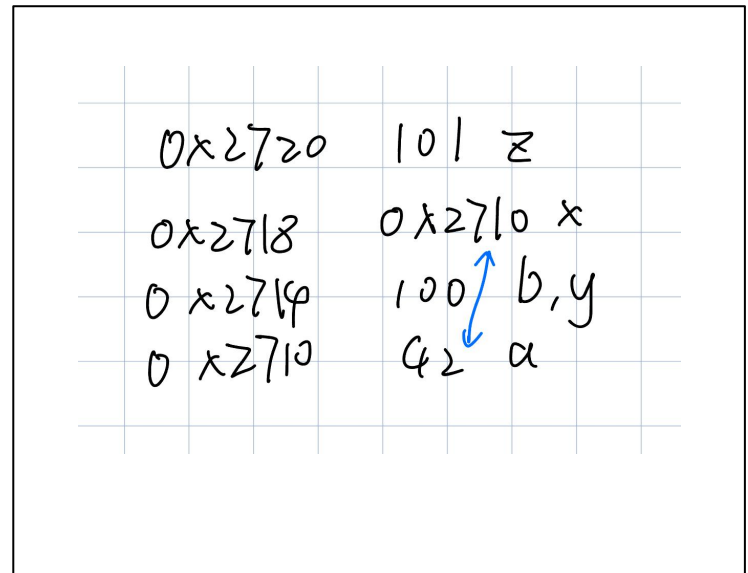
expression	type	value
&a	int*	0x2710
&b	double*	0x2714
*c	int	10
*&b	double	3.14

expression	type	value
d	int	c
**d	int	10
&a == &e	boolean	true
&*d	int**	0x271c

2.1.4 Memory Diagrams

In the box, draw a diagram showing the state of memory for the program at the point indicated below.

```
int main(){
    int a = 42;
    int b = 100;
    int *x = &a;
    int &y = b;
    int z = y;
    z++;
    // Draw memory at this point
}
```



You've reached the end of checkpoint 1! Let your instructor know you're ready to check your work!

2.1.5 Makefile Practice

Typing out compilation commands works for simple programs, but it can become tedious as projects grow. To automate compilation, developers use `make` alongside a Makefile to define build rules, dependencies, and cleanup routines.

Open the Makefile for [Project 1](#) and find the rule that cleans the project directory. Examine the Makefile and answer the questions below:

What is the name of this target?	<code>clean</code>
What commands would you run in your terminal to execute the target?	<code>Make clean</code>
Does this target list any dependencies? If so, what are they?	<code>none</code>
What underlying shell command runs when this target is invoked?	<code>rm -rvf *.exe *~ *.out *.dSYM *.stackdump</code>
What files are being removed when the target is invoked?	<code>Executables, backup files, output files, debug, stack dump</code>

2.2.1 Unit Testing Framework Tutorial

In project 1, you wrote unit tests to check for expected behavior using `assert`. However, this approach can be a bit clunky - the very first failed `assert` crashes out of the program and the rest of your tests don't even run.

Starting with project 2 and beyond, you'll use a **unit testing framework** instead, which provides special testing assertions (e.g. `ASSERT_EQUAL`). Instead of crashing on failure, the framework records each test success/failure and reports overall results at the end.

As a group, work through our [Unit Testing Framework Tutorial](#) (also linked from the tutorials section of eecs280.org). In brief, the tutorial will have you:

- Download `tutorial.hpp` and `tutorial.cpp` starter files with buggy code.
- Learn some new syntax and strategies for testing.
- Create tests in `tutorial_tests.cpp` to catch bugs.
- Submit your tests to the autograder, which evaluates their thoroughness.

When you're finished, paste a copy of your `tutorial_tests.cpp` on the next page.

Note: You do not need to get a perfect score on the autograder. It is sufficient enough to earn some points on the autograder.

2.2.1 Unit Testing Framework Tutorial (Continued)

Paste a copy of your `tutorial_tests.cpp` into the box below. Use tiny font so it fits :D
The tutorial has you submit to the autograder, but it's alright if you didn't necessarily catch all the bugs there. You'll get credit for the lab as long as you've written some tests.

```
#include <vector>
#include "tutorial.hpp"
#include "unit_test_framework.hpp"

using namespace std;

TEST(test_slide_right_1) {
    vector<int> v = { 4, 0, 1, 3, 3 };
    vector<int> expected = { 3, 4, 0, 1, 3 };
    slideRight(v);
    ASSERT_SEQUENCE_EQUAL(v, expected);

    v = {5};
    expected = {5};
    slideRight(v);
    ASSERT_SEQUENCE_EQUAL(v, expected);

    v = {1, 2};
    expected = {2, 1};
    slideRight(v);
    ASSERT_SEQUENCE_EQUAL(v, expected);
}

TEST(test_flip_1) {
    vector<int> v = { 4, 0, 1, 3, 3 };
    vector<int> expected = { 3, 3, 1, 0, 4 };
    flip(v);
    ASSERT_SEQUENCE_EQUAL(v, expected);

    v = {1};
    expected = {1};
    flip(v);
    ASSERT_SEQUENCE_EQUAL(v, expected);

    v = { 4, 0, 3, 3 };
    expected = { 3, 3, 0, 4 };
    flip(v);
    ASSERT_SEQUENCE_EQUAL(v, expected);
}

TEST_MAIN() // No semicolon!
```

You've reached the end of a code review exercise! Please have one member of your group submit your responses here: <https://forms.gle/VPALQPtgW7JPeqqT8>

2.2.2 Unit Testing Framework Tutorial

Which of the following are true about the UTF? Write "true" or "false" in the box.

F	A benefit of the UTF is that it stops testing on the first failed assertion.
T	Each test file must have one <code>TEST_MAIN()</code> call at the bottom of the file.
T	The UTF has special assertions, such as <code>ASSERT_EQUAL()</code> and <code>ASSERT_SEQUENCE_EQUAL()</code> . Regular <code>assert()</code> should not be used.
T	The autograder gives points for each bug your test cases can catch, but it will first discard any tests that fail to pass on a correct solution.

Briefly explain when the `ASSERT_ALMOST_EQUAL()` assertion must be used.

This is for decimal precision. When comparing doubles or large decimals, this is needed.

You've reached the end of checkpoint 2! Let your instructor know you're ready to check your work!